Predicting Property Price Category Using Machine Learning

A Comparative Evaluation of Classical and Deep Learning Classifiers

CSE422 — Artificial Intelligence

BRAC University

Md. Tanvirul Islam Rifat

Student ID: 22101311

GitHub: tanvirul-islam-rifat

1. Executive Summary

This report documents an end-to-end supervised machine learning pipeline built to classify residential flats (apartments) into three price categories — Low, Medium, and High — using twelve structural and locational features. The pipeline covers exploratory data analysis, missing-value and outlier handling, feature encoding and scaling, and a comparative evaluation of six models: Decision Tree, K-Nearest Neighbors, Gaussian Naive Bayes, Random Forest, XGBoost, and a fully connected neural network.

The central finding is not a high-performing model, but a well-supported negative result: every model tested, across two very different model families (tree/distance/probabilistic classical ML and a multi-layer neural network), converges to the same performance ceiling of roughly 32-38% accuracy on a balanced three-class problem, i.e. only marginally above the 33.3% random-guessing baseline. Correlation analysis and per-feature boxplots independently support the same conclusion: the available features carry very little signal about the assigned price category in this dataset. This report treats that as a legitimate, reportable outcome rather than a failure, and discusses what it implies about the dataset and about model selection in low-signal settings.

2. Problem Statement

Real-estate platforms commonly need to auto-tag new listings with a price bracket (e.g. Low / Medium / High) for search filters, recommendation ranking, or fraud/anomaly flags (a listing priced far outside its predicted bracket may warrant review) before a final price is confirmed by an agent. This project frames that as a supervised, multi-class classification problem: given structural attributes of a flat (size, bedrooms, bathrooms, floor, age) and locational attributes (location tier, distance to city center, nearby schools, security level, parking, balcony), predict the pre-assigned Price_Category label.

The task is deliberately treated as model comparison under real-world data conditions — substantial missingness, mixed categorical/numerical features, and no guarantee that the features chosen are actually predictive — rather than as a curated, guaranteed-to-work textbook dataset.

3. Dataset Description

The dataset contains 1,200 flat records and 12 columns (11 features plus the target label), loaded from a CSV file.

3.1 Feature Schema

Column	Type	Description
Location	Categorical	City Center / Suburbs / Countryside
Size_sqft	Numeric	Floor area in square feet
Num_Bedrooms	Numeric	Bedroom count
Num_Bathrooms	Numeric	Bathroom count
Has_Balcony	Binary	Yes / No
Floor_Number	Numeric	Floor the unit is on
Building_Age_Years	Numeric	Age of the building
Parking_Available	Binary	Yes / No
Nearby_Schools	Categorical	Few / Many
Distance_to_CityCenter_km	Numeric	Distance in kilometers
Security_Level	Categorical	Low / Medium / High
Price_Category (target)	Categorical	Low / Medium / High

3.2 Class Balance

The target label is close to perfectly balanced across its three classes, which rules out class imbalance as an explanation for the modeling results discussed later:

Class	Count	Share
Medium	413	34.4%
Low	402	33.5%
High	385	32.1%

4. Exploratory Data Analysis

4.1 Missing Data

Every feature column except the target had a substantial fraction of missing values (roughly 8%-12% of rows per column), while the target itself had none:

Column	Missing	Column	Missing
Location	140	Parking_Available	101
Size_sqft	106	Nearby_Schools	124
Num_Bedrooms	136	Distance_to_CityCenter_km	103
Num_Bathrooms	143	Security_Level	126
Has_Balcony	121	Price_Category	0
Floor_Number	132		
Building_Age_Years	127		

4.2 Outlier Screening

A column-wise Z-score check (threshold $|z| > 3.0$) was run across all numeric features. It found zero outliers in every column, so no rows were removed on outlier grounds.

4.3 Feature Correlation

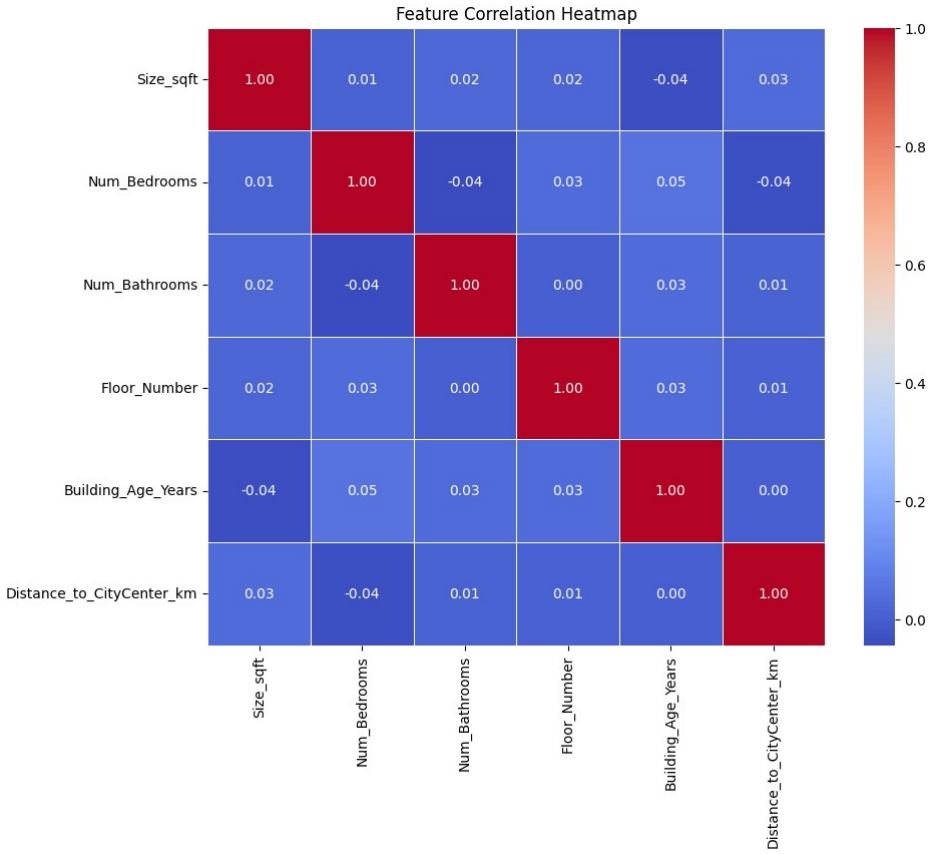

Figure 1. Pearson correlation heatmap across numeric features. All pairwise correlations are weak, and none of the numeric features correlates meaningfully with the encoded target.

4.4 Class Distribution

Figure 2. Price_Category is balanced across Low, Medium, and High.

Per-feature boxplots of each numeric variable against Price_Category (generated in the accompanying notebook) showed no strong visual separation between classes for any single feature; Num_Bathrooms showed a very slight positive association with price category, but not one strong enough to be discriminative alone. Combined with the correlation heatmap, this was the first indication that this would be a low-signal classification problem.

5. Data Preprocessing and Feature Engineering

- **Deduplication:** Checked for and removed exact duplicate rows (none were found; row count remained 1,200).
- **Numeric imputation:** Numeric columns (Size_sqft, Num_Bedrooms, Floor_Number, Num_Bathrooms, Building_Age_Years, Distance_to_CityCenter_km) had missing values imputed with the column mean.
- **Categorical imputation:** Categorical columns (Location, Has_Balcony, Parking_Available, Nearby_Schools, Security_Level) had missing values imputed with the column mode.
- **Multi-class encoding:** Location, Nearby_Schools, Security_Level, and the target Price_Category were label-encoded (scikit-learn LabelEncoder) for use with tree- and distance-based models.
- **Binary encoding:** Has_Balcony and Parking_Available were mapped Yes → 1, No → 0.
- **Train/test split:** 70/30 train/test split, stratified on Price_Category, random_state=42, to preserve class balance in both splits.
- **Scaling:** MinMaxScaler fit on the training set and applied to both splits, feeding the classical models scaled features in [0, 1].

6. Modeling Methodology

Six models spanning four different learning paradigms were trained on the identical preprocessed train/test split, so that results are directly comparable:

- **Model 1 —** Decision Tree Classifier (scikit-learn, default hyperparameters, random_state=42).
- **Model 2 —** K-Nearest Neighbors Classifier (k = 5).
- **Model 3 —** Gaussian Naive Bayes.
- **Model 4 —** Random Forest Classifier (default hyperparameters, random_state=42).
- **Model 5 —** XGBoost Classifier (log-loss objective, random_state=42).
- **Model 6 —** A Keras Sequential neural network: Dense(64, ReLU) → Dropout(0.5) → Dense(32, ReLU) → Dropout(0.3) → Dense(3, softmax); Adam optimizer, sparse categorical cross-entropy, 100 epochs, batch size 32.

For the five classical models, each was evaluated on the held-out test set using Accuracy, macro-averaged Precision, Recall, and F1-score (macro averaging was used deliberately, since the classes are balanced but treating them asymmetrically would hide weak per-class performance), and multiclass ROC-AUC using the one-vs-rest scheme. The neural network was evaluated with test accuracy and a train/validation loss curve, following a slightly different workflow in the source notebook.

7. Results

7.1 Classical Model Comparison

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Decision Tree	0.3833	0.3841	0.3831	0.3831	0.5375
XGBoost	0.3611	0.3597	0.3600	0.3598	0.5209
Gaussian Naive Bayes	0.3500	0.3453	0.3478	0.3452	0.4957
Random Forest	0.3444	0.3442	0.3436	0.3439	0.5019

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
KNN (k=5)	0.3167	0.3152	0.3196	0.3068	0.4684

Decision Tree posted the highest macro F1-score (0.3831), narrowly ahead of XGBoost. All five ROC-AUC values sit close to 0.50 (the value expected from random class discrimination), with Decision Tree the only model to move meaningfully away from it (0.5375).

7.2 Confusion Matrices — Classical Models

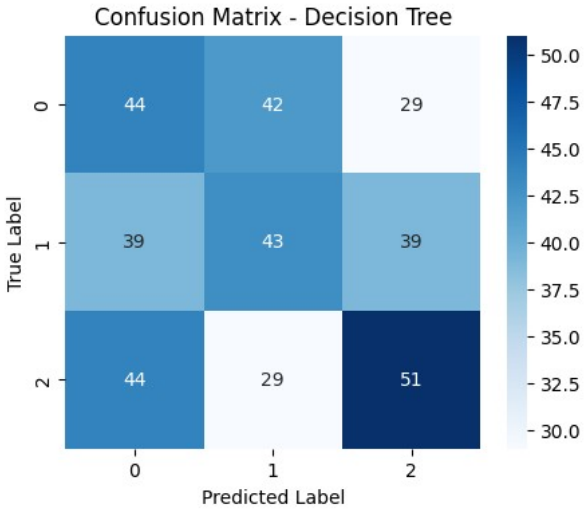

Figure 3. Confusion matrix — Decision Tree.

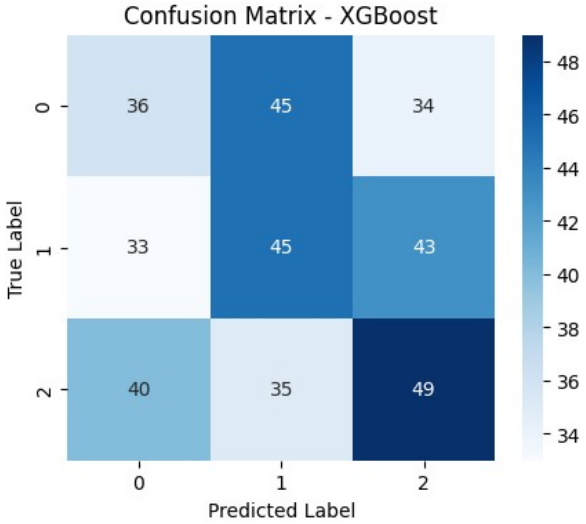

Figure 4. Confusion matrix — XGBoost.

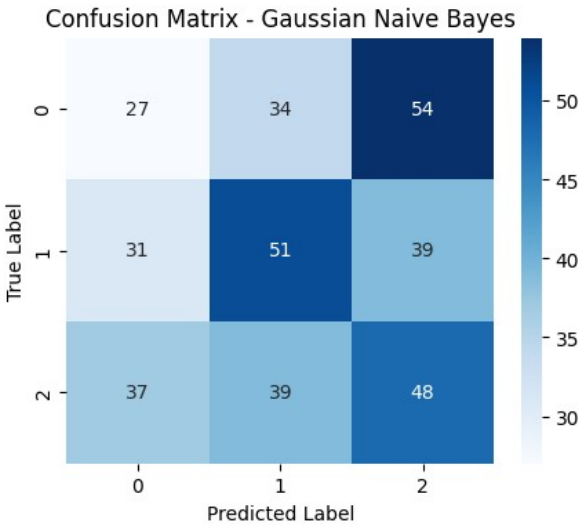

Figure 5. Confusion matrix — Gaussian Naive Bayes.

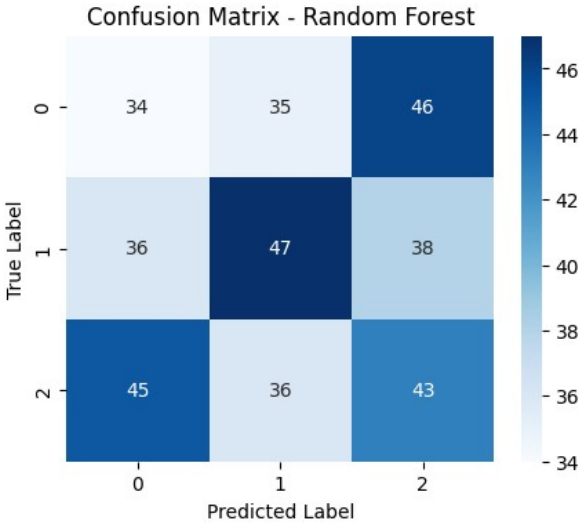

Figure 6. Confusion matrix — Random Forest.

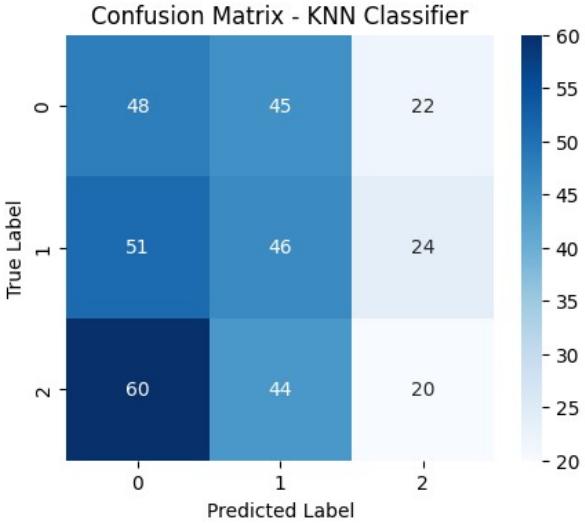

Figure 7. Confusion matrix — K-Nearest Neighbors.

Across all five models, misclassifications are spread broadly across all three classes rather than concentrated in one confusable pair — the signature of a model that has not learned a strong decision boundary, as opposed to one confusing only two adjacent categories (e.g. Medium vs. High).

7.3 Model Ranking by F1-Score

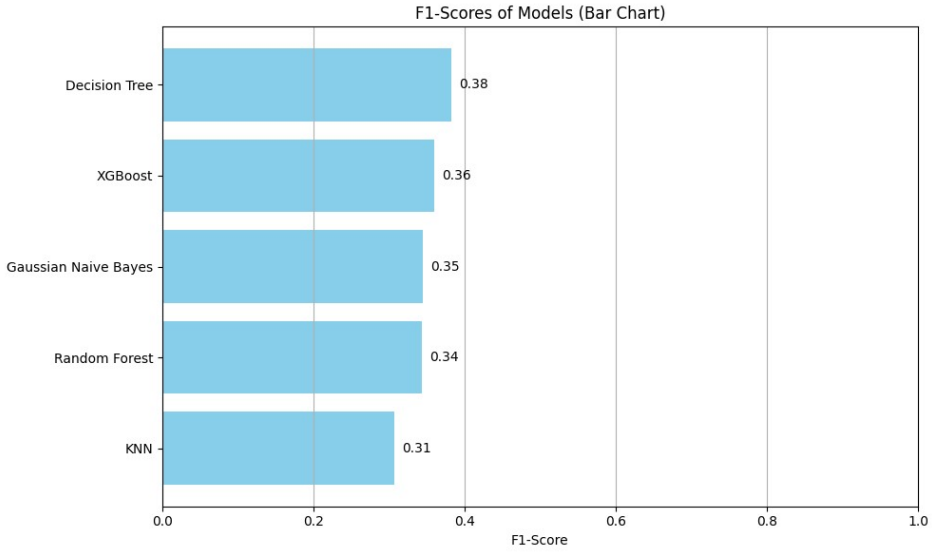

Figure 8. Macro F1-score ranking across all five classical models.

7.4 Neural Network

The neural network was trained for 100 epochs. Training loss fell sharply within the first ~10 epochs while validation accuracy plateaued almost immediately around 32-36%, and training loss continued to fall far below validation loss in later epochs — a classic overfitting signature in which the network memorizes patterns in the training split that do not generalize, rather than learning a transferable relationship between features and price category.

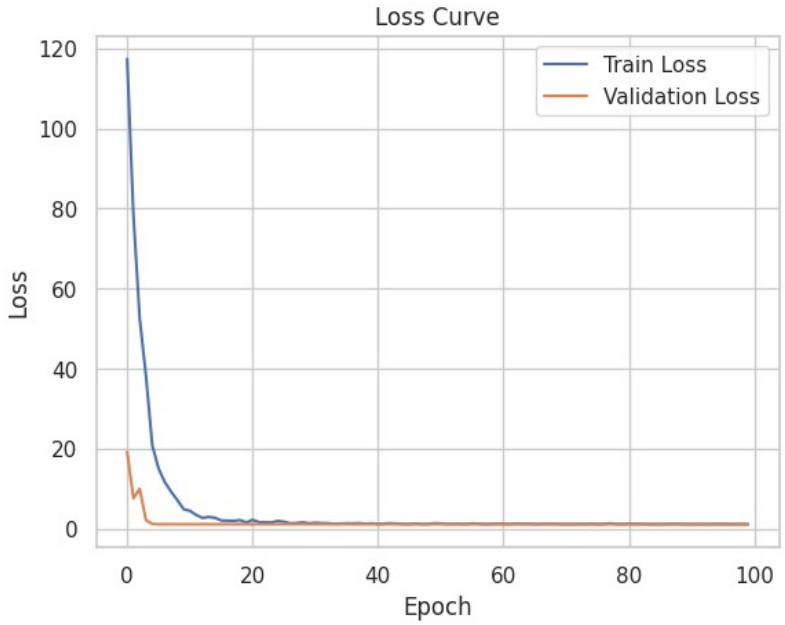

Figure 9. Training vs. validation loss over 100 epochs.

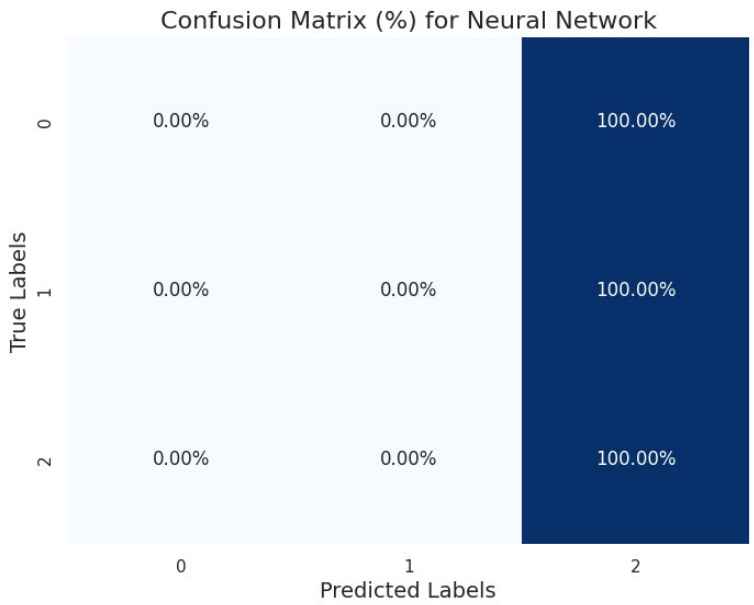

Figure 10. Neural network confusion matrix (row-normalized, %).

Final test accuracy: 34.44% — within the same narrow band as every classical model, despite the network having far more capacity and being the only model to show clear overfitting.

8. Discussion and Key Findings

- **Convergent low performance across model families:** Every model, regardless of family — a single tree, an ensemble of trees, a distance-based method, a probabilistic model, a gradient-boosted ensemble, and a neural network — lands within a 6-point accuracy band (31.7%-38.3%), only just above the 33.3% baseline for a balanced 3-class problem. When six structurally different learners agree this closely, the bottleneck is very unlikely to be model choice.
- **EDA predicted this outcome:** This matches the EDA in Section 4: the correlation heatmap showed weak numeric-feature correlations, and per-feature boxplots showed no visual separation between classes. The modeling stage did not discover a weak-but-present signal that EDA missed — it confirmed the absence flagged during EDA.
- **More model capacity did not help:** The neural network is the only model that clearly overfits (falling training loss, flat validation performance), yet it does not outperform simpler models on held-out data. Extra model capacity cannot manufacture a signal that is not present in the underlying features.
- **Practical implication:** Given a synthetic or heavily engineered feature set, it is plausible that Price_Category was assigned independently of the listed features (e.g. randomly, or from an unobserved variable not included in this dataset). Reporting a near-random ceiling honestly is more useful to a reader than presenting a marginal 38% accuracy as if it were a meaningful result.

9. Limitations and Future Work

- **Hyperparameter search:** No hyperparameter tuning (grid/random search) was performed for the classical models; default parameters were used throughout for a fair, apples-to-apples first comparison.
- **Imputation strategy:** Mean/mode imputation was used for simplicity. Given ~10% missingness per column, a more sophisticated imputer (KNNImputer or MICE) could be tried, though it is unlikely to fix a signal problem at the feature level.
- **Feature availability:** If further data collection were possible, engineered or interaction features (e.g. price-per-square-foot proxies, neighborhood-level aggregates) or additional real-world variables (amenities, renovation status, exact

address-level location data) would be the highest-leverage next step, since the current features do not appear to separate the classes.

- **Baseline comparison:** An explicit majority-class and stratified-random baseline was not computed programmatically in the notebook; it is included in this report analytically (33.3% for three balanced classes) and would be a natural first cell to add.

10. Conclusion

This project implemented a complete classification pipeline — EDA, missing-data and outlier handling, encoding, scaling, and a six-model comparison spanning classical machine learning and deep learning — to predict flat price category from structural and locational features. The headline result is that no model, however different its underlying approach, exceeded roughly 38% accuracy on a balanced three-class task, converging just above the 33.3% random baseline. Correlation analysis and boxplots independently support the same conclusion. The project's value lies in demonstrating a rigorous, multi-model evaluation methodology and in correctly recognizing and reporting a genuine low-signal result rather than overstating marginal gains.

Appendix A — Environment and Reproducibility

- **Libraries:** Python 3, pandas, numpy, scikit-learn, xgboost, tensorflow/keras, matplotlib, seaborn, scipy.
- **Split:** 70% train / 30% test, stratified split, random_state = 42 throughout for reproducibility.
- **Dataset size:** 1,200 rows, 12 columns, provided as flat_price_dataset.csv in this repository's data/ folder.
- **Full code:** flat_price_classification.ipynb (this project's companion Jupyter notebook) contains the full, runnable code for every step described in this report.